

Algoritmi e Strutture Dati – Parte A – 28/08/26

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Si consideri la seguente procedura, applicata a un vettore A di n elementi.

```

int mystery(int[] A, int n)
{
    int s = 0
    for i = 0 to n-2 do
        int j = i + 1
        while j < n and A[j] < A[i] do
            s = s + 1
            j = j + 1
    return s
}

```

Determinare la complessità nel caso pessimo e nel caso ottimo, motivando la risposta.

A2 – Cavallo su scacchiera – Punti ≥ 10

Sia data una scacchiera $n \times n$, con $n \geq 1$, le cui righe e colonne sono numerate da 0 a $n - 1$. Un cavallo che si trova nella casella (r, c) può spostarsi in una casella (r', c') della scacchiera se

$$(|r' - r|, |c' - c|) \in \{(1, 2), (2, 1)\}.$$

Sono date una casella sorgente $(startRow, startCol)$ e una casella destinazione $(endRow, endCol)$, entrambe appartenenti alla scacchiera.

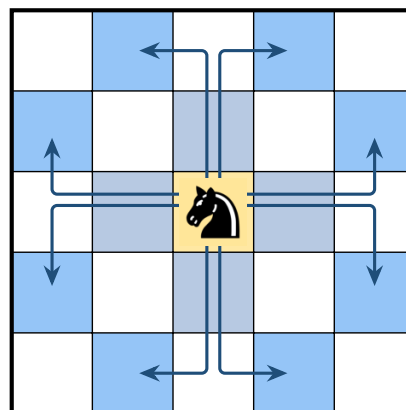
Scrivere un algoritmo

```

int minKnightMoves(int n, int startRow, int startCol,
                   int endRow, int endCol)

```

che restituisca il numero minimo di mosse necessarie per raggiungere la destinazione dalla sorgente, oppure -1 se la destinazione non è raggiungibile. Se sorgente e destinazione coincidono, l'algoritmo deve restituire 0.



Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

A3 – Sottoalbero ABR massimo – Punti ≥ 12

Sia dato un albero binario T , non necessariamente un albero binario di ricerca, in cui ogni nodo t contiene una chiave intera $t.key$. Un sottoalbero di T è costituito da un nodo di T e da tutti i suoi discendenti.

Un sottoalbero è un *albero binario di ricerca* (ABR) se, per ogni suo nodo t , tutte le chiavi del sottoalbero sinistro sono strettamente minori di $t.key$ e tutte le chiavi del sottoalbero destro sono strettamente maggiori di $t.key$.

Scrivere un algoritmo

```

int largestBST( $TREE\ T$ )

```

che restituisca il numero di nodi del più grande sottoalbero di T che sia un ABR. Se T è vuoto, l'algoritmo deve restituire 0.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

Algoritmi e Strutture Dati – Parte B – 28/08/26

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

B1 – Numeri inquadrabili – Punti ≥ 8

Due interi si dicono *inquadrabili* se la loro somma è pari al quadrato di un intero; ad esempio, 4 e 5 sono inquadrabili, in quanto $4 + 5 = 3^2$.

Descrivere un algoritmo che, dati due vettori X e Y contenenti complessivamente $2n$ valori interi tutti distinti, restituisca il maggior numero di coppie inquadrabili (x, y) , con $x \in X$ e $y \in Y$, assumendo che ogni valore possa essere utilizzato al più una volta.

Ad esempio, se $X = [6, 1, 15]$ e $Y = [10, 3, 7]$, il maggior numero di coppie inquadrabili è 2, dato da $1 + 3 = 2^2$ e $15 + 10 = 5^2$. Il fatto che siano anche presenti $6 + 3 = 3^2$ e $6 + 10 = 4^2$ non permette di ottenere una terza coppia, perché 3 e 10 sono già stati usati nelle altre due.

Discutere brevemente la correttezza della soluzione proposta e calcolare la complessità computazionale.

B2 – Dismutazioni – Punti ≥ 10

Una *dismutazione* π dell'insieme $\{0, \dots, n-1\}$, con $n \geq 0$, è una permutazione in cui nessun elemento è nella posizione "corretta", ovvero $\pi[i] \neq i$, per $0 \leq i < n$. Scrivere un algoritmo

`printDismutations(int n)`

che stampi tutte le dismutazioni dell'insieme $\{0, \dots, n-1\}$.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, le dismutazioni di $\{0, 1, 2\}$ sono $\{1, 2, 0\}$ e $\{2, 0, 1\}$. Invece, $\{0, 2, 1\}$ non è una dismutazione perché 0 è nella posizione corretta.

B3 – Pari-bilanciato – Punti ≥ 12

Un sottovettore *contiguo* di un vettore A si dice *pari-bilanciato* se il numero di elementi pari è uguale al numero di elementi dispari contenuti in esso.

Dato un vettore di interi A di lunghezza $n \geq 0$, scrivere un algoritmo

`int maxBalanced(int[] A, int n)`

che restituisca la lunghezza massima di un sottovettore pari-bilanciato contenuto in A .

Se non esiste alcun sottovettore bilanciato, l'algoritmo deve restituire 0.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, se $A = [2, 4, 1, 6, 8, 3, 5]$, il sottovettore $A[1 \dots 6] = [4, 1, 6, 8, 3, 5]$ è pari-bilanciato e ha lunghezza massima 6.